SYNTHAGENT: Reliable LLM-Based Synthetic Data Generation for Small Healthcare Datasets

Sancho Amaral Simões[1,*], João P. Vilela[1,2], Pedro Henriques Abreu[1]
[1]*University of Coimbra, CISUC/LASI – Centre for Informatics and Systems of the University of Coimbra, Department of Informatics Engineering*, Coimbra, Portugal
[2]*CRACS/INESCTEC-DCS - FCUP, University of Porto*, Porto, Portugal
[*]Corresponding author
sanchosimoes@student.dei.uc.pt, jvilela@fc.up.pt, pha@dei.uc.pt

***Abstract*—Prediction tasks in healthcare, such as diagnosing a disease or anticipating an adverse outcome, often have to be learned from small datasets, because the disease is rare or because labelling records is expensive. Synthetic data generation can be a solution for such contexts, but established generators, such as oversampling techniques and generative adversarial networks, learn only from the available records, recombining patterns that already exist in the data. Large language models (LLMs) can address this issue by adding domain knowledge acquired during pre-training, without being trained on the target data. When prompted directly, however, LLMs often return fewer records than requested, malformed records or values of the wrong type, so their output cannot be used as a dataset. To address this issue, we present SYNTHAGENT, a synthetic data generation system for small healthcare datasets that combines an LLM with a control layer. Inferring the columns and their types from the real data, the control layer checks every response of the LLM, asks for the correction of invalid values and requests new records until the configured number is reached, while recording the cost of every request. In an evaluation on six public healthcare datasets with LLMs from two providers, SYNTHAGENT completed ~~the synthetic datasets~~ every synthetic dataset to the configured size, equal to the number of real training records, which direct prompting of the same LLM did not achieve in any run, even when its responses were forced into a fixed format, at a small additional cost ~~. Added to the training data of ten classifiers , its records~~ in tokens. When ten classifiers were trained with and without synthetic records, the records of SYNTHAGENT yielded the largest average F1-score gain among the generators compared and avoided the losses of up to 10.6 points that SMOTE, CTGAN and TVAE caused on individual datasets.**

***Index Terms*—small data, synthetic data generation, large language models, healthcare**

I. INTRODUCTION

Many prediction tasks in healthcare, such as diagnosing a disease or anticipating an adverse outcome, suffer from a lack of data. A rare disease may have only a few dozen diagnosed cases, a study conducted in a single healthcare facility is limited to the patients it can recruit, and labelling records often requires expert review. Data sharing between facilities could increase the number of records, but it is frequently restricted by privacy regulations [1], so each facility often has to learn from its own data. In our work we refer to such datasets as *small data*: datasets whose number of records is too low to train a reliable prediction model, a limit that depends on the number of features, the number of classes and the frequency of the outcome [2]. A common way to mitigate small data, and a lack of data more generally, is synthetic data generation (SDG), which adds artificial records that follow the distribution of the real ones to the training set [3]. This paper focuses on SDG for prediction tasks on small healthcare datasets.

SDG is, however, hardest when data are small. Oversampling techniques, such as the Synthetic Minority Oversampling Technique (SMOTE) [4], and deep generative models, such as the conditional tabular generative adversarial network (CTGAN) and the tabular variational autoencoder (TVAE) [5], learn the distribution from the same few records they are meant to complement, recombining patterns that already exist in those records, for example by interpolating between two existing patients ~~[6]~~ [6, Sec. 2.8]. Large language models (LLMs) offer a new perspective to address such issue. They are pre-trained on large text corpora, which gives them domain knowledge about healthcare variables, such as plausible ranges and typical relations between them, and they can generate new records from a few real examples placed in the prompt, without any training on the target dataset [7], [8]. Their output, however, is ~~free text rather than~~ text that has to be parsed into records, and nothing guarantees that it forms a dataset: a response may contain fewer records than requested, malformed records, missing or invented columns, or values of the wrong type (Figure 1). Every augmentation method is configured with the number of synthetic records to add, usually expressed as a proportion of the real training set [4], and a generator that silently returns fewer or malformed records changes the training data without the user noticing. In addition, each request to a commercial LLM has a monetary cost, proportional to the number of tokens (word fragments) processed, which is rarely reported.

One problem that stands out is therefore how to obtain from an LLM a synthetic dataset with the configured number of records, the columns and types of the real data and a known cost, so that it can be used to improve the classifiers trained on small data. Formally, given the training records D of a small dataset, their columns and types S and the configured number of synthetic records N, a configuration θ of the generator produces a synthetic dataset G_θ. The goal is to find a configuration whose dataset belongs to the set $\mathcal{F}(S, N)$ of datasets with exactly N records that conform to

S and that maximizes the gain in classifier performance ΔU, while keeping the distance from the real distribution L_{fid}, the privacy risk L_{priv} and the cost κ low:

$$\theta^{\star} \in \underset{\theta:\, G_\theta \in \mathcal{F}(S,N)}{\arg\min} \big(-\Delta U,\ L_{\text{fid}},\ L_{\text{priv}},\ \kappa\big)(\theta). \qquad (1)$$

The constraint follows the data-quality dimensions of completeness and conformity [9], [10], and the objectives are the utility, fidelity and privacy on which synthetic data are evaluated [3], [11], together with cost: ΔU is the change in classifier performance when G_θ is added to D, L_{fid} the divergence between the real and synthetic distributions, L_{priv} the success of a membership-inference attack, and κ the number of tokens processed; Section V defines how each is measured. The objectives are kept separate rather than combined with weights, because the right trade-off depends on the use of the data.

Currently, deep generators [5], [12], [13] and language models fine-tuned on the target data [7], [14] must be trained on the scarce records, and they were designed and evaluated on datasets with tens of thousands of records. Approaches that prompt an LLM directly, such as CLLM [8] and EPIC [15], avoid training, but they focus on the design of the prompt and on the selection of the generated records; they do not ensure that the output has the configured number of records and the correct columns and types, and they do not report its cost. Finally, generators are usually compared on the result of a single run, which is unreliable in small data. A small test set amplifies chance: with 40 test records, one misclassified record changes accuracy by 2.5 percentage points, and the randomness of the data split and of the LLM's sampling makes two runs of the same generator differ by amounts comparable to the differences being compared.

To address these limitations, we propose SYNTHAGENT (*synthetic data agent*), an SDG system for small healthcare datasets that combines a pre-trained LLM, which proposes new records, with a *control layer*, a deterministic software component that checks every response of the LLM before any of its records is accepted into the synthetic dataset. The control layer selects the real records shown to the model, repairs and validates every response and asks for corrections, requests exactly the missing records, and logs the cost of every request. Like an LLM agent, the system thus alternates actions, the requests to the LLM, with observations, the checks of their output, which gives the system its name. This point is crucial since it makes the domain knowledge of LLMs usable where established generators are weakest, and since each healthcare facility can run the system on its own data and obtain a dataset that can be audited like any other data source.

The main contributions of this paper are:

- SYNTHAGENT, an SDG system for small healthcare datasets that requires no training on the target data and, through its control layer, delivers the configured number of records with the columns and types of the real data and a recorded cost;
- a deployment pattern in which each healthcare facility runs SYNTHAGENT locally and only model updates are shared, so that no record leaves the facility;
- an open-source implementation that works with any OpenAI-compatible LLM endpoint, including locally served models.

Our objective is to determine whether SYNTHAGENT makes LLM-based SDG a reliable and useful option for small healthcare datasets. To this end, four main research questions emerged in this work:

RQ1 Is there any relationship between the control layer and the reliability of the generated datasets?

RQ2 ~~Are~~ Do the generated records ~~faithful to~~ preserve the distribution of the real data without reproducing ~~them~~the real records?

RQ3 Do the generated records improve the performance of classifiers trained on small data?

RQ4 How does SYNTHAGENT compare with established generation strategies, and how much do its results depend on the LLM?

To answer these questions, an experimental setup was built with six public healthcare datasets containing between 195 and 768 records. For each dataset, synthetic records were produced by SYNTHAGENT with seven commercial LLMs from two providers, by direct prompting of the same LLMs, with and without the provider's option to force responses into a fixed format, and by three established generation strategies, SMOTE, CTGAN and TVAE. Each dataset was randomly divided ten times into a training and a test set, and on each division ten classifiers were trained with and without synthetic records and compared on the same test set. The main conclusions are the following. Without the control layer, the LLM never returned the configured number of records, even when its responses were forced into the format of the data, whereas SYNTHAGENT completed every dataset at only 1.9% more cost (RQ1). The synthetic records reproduced the distribution of each real feature with a fidelity of 73 out of 100, and a membership-inference attack could not reliably identify the real records used, although a classifier could still tell synthetic from real records (RQ2). The effect on classifiers depended on the dataset and the classifier, and with the strongest LLMs the average F1-score increased by 1.1 to 2.4 percentage points (RQ3). SYNTHAGENT gave the largest and most stable gain among the generators compared: synthetic records can also harm a classifier when they misrepresent the real data, and the established generators caused losses of up to 10.6 points on individual datasets, against at most 0.53 points for SYNTHAGENT with GPT-4.1; only the choice of LLM changed the results by more than the variation between repeated runs (RQ4).

The remainder of the paper is organized as follows. Section II presents the background knowledge~~and~~ , Section III describes SYNTHAGENT and Section IV its deployment across healthcare facilities. Section V presents the experimental setup, Section VI reports the results, Section VII discusses their

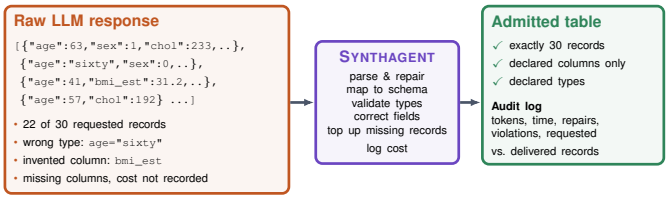


Fig. 1. The problem addressed in this paper, in an illustrative example. Asked for 30 records of a heart-disease dataset, an LLM response (left) can contain too few records, invented or missing features and values of the wrong type. SYNTHAGENT (right) validates and corrects every response, so the resulting dataset has the requested size and schema, and a log records each correction and its cost.

implications and limitations, and Section VIII concludes the paper.

II. BACKGROUND KNOWLEDGE

In this section, the families of methods for synthetic data generation (SDG) of tabular data are first reviewed. Then, the limitations of each family when data are small are discussed, and finally the work on validating generated data and on evaluating synthetic data, on which SYNTHAGENT builds, is presented.

A. Synthetic Data Generation for Tabular Data

Tabular SDG methods fall into four families, which differ in how they produce new records. *Interpolation and statistical methods* derive new records directly from the real ones or from a simple probabilistic model of them: SMOTE places a new minority-class record on the line segment between two neighbouring records of that class [4], and Bayesian networks estimate the dependencies between features and sample new records from them [16]. *Deep generative models* train a neural network to approximate the joint distribution of the features and then sample from it: CTGAN trains a generator against a discriminator, TVAE learns a compressed representation of the records [5], and diffusion models such as TabDDPM, CoDi and TabSyn learn to reverse a gradual corruption of the records [12], [13], [19]; relational and cross-silo variants extend these models to several tables or institutions [20], [21]. *Fine-tuned language models* write each record as a sentence, fine-tune a language model on these sentences and generate new ones, as in GReaT [7], REaLTabFormer [14] and TabLoft [22]. *Prompted LLMs* place a few real records in the prompt of a pre-trained LLM and ask it for new ones, without any training, as in CLLM [8] and EPIC [15].

B. Synthetic Data Generation in Small Data

Most of the previously mentioned generators were designed and benchmarked on datasets with tens or hundreds of thousands of records, such as Adult (48,842 records) or Covertype (581,012 records) [5], and their behaviour on a few hundred records has received far less attention. Table I summarizes, for each family, how records are produced and its main limitation when data are small. Interpolation and statistical methods, as well as deep generators, are fitted only to the real records, so they cannot add information beyond what those records contain ~~[6]~~ [6, Sec. 2.8]. They also treat every column as an anonymous number: they do not know that a column holds a patient's age or cholesterol level, and therefore cannot use what is known about these variables. Deep generators and fine-tuned language models have many parameters by design, which a few hundred records cannot estimate reliably, so they may produce records outside the observed ranges and types, or reproduce training records [23]. Prompted LLMs need no training and bring domain knowledge, but nothing ensures that their output contains the configured number of records with the correct columns and types, and the cost of obtaining it is not reported; CLLM [8] filters the generated records by their learning dynamics and EPIC [15] designs balanced prompts, but both concern the quality of the records that are kept rather than the size, structure and cost of the output. As discussed in Section I, all families are also usually compared on a single run. SYNTHAGENT keeps the advantages of prompted LLMs and adds the control layer that they lack, and its evaluation repeats every experiment to measure the variation between runs.

C. Validation of Generated Data

The control layer applies to generation a principle long used in data management: data produced by an automatic process are checked against explicit constraints before they are accepted. Kamino enforces integrity constraints during private data synthesis [24], data-cleaning systems combine statistical inference with integrity constraints [25], [26], and recovering structured records from the free-form output of an LLM has been studied as a data-engineering problem [27]. Constrained decoding goes further and restricts the tokens an LLM may emit to those allowed by a grammar or a JSON schema [17], [18], and LLM providers now offer it as a structured-output mode. It guarantees the format of each response, but not how many records a response contains, so it does not by itself deliver a dataset of a given size; Section VI-A tests it as a baseline. In SYNTHAGENT, the constraints are the columns and types inferred from the real data, and the checks are applied to every response of the LLM, in the action–observation loop of LLM agents [28], [29].

D. Evaluating Synthetic Data

Data-preparation studies have shown that a step that looks beneficial can harm downstream models [30], [31], and evaluation frameworks such as Synthcity and SynthEval recommend assessing fidelity, privacy and utility together [3], [11]. Fidelity can be complemented by a classifier two-sample test, which asks whether a classifier can tell real from synthetic records [32], and privacy by membership-inference attacks, which ask whether an attacker can tell which records were used for generation [23], [33]. Formal guarantees require syntactic anonymization models such as ℓ-diversity and t-closeness [34], [35] or differential privacy [36], which SYNTHAGENT does not claim.

TABLE I

FAMILIES OF TABULAR SDG METHODS: HOW THEY PRODUCE RECORDS, THEIR MAIN LIMITATION IN SMALL DATA, AND THEIR PROPERTIES. ✓: PROVIDED BY DESIGN; ×: NOT PROVIDED. SMOTE'S NUMBER OF RECORDS IS SET BY THE CLASS BALANCE, NOT BY THE USER; FINE-TUNED LANGUAGE MODELS DISCARD UNREADABLE RECORDS BUT DO NOT CHECK TYPES; CONSTRAINED DECODING IS A VARIANT OF PROMPTED LLMS.

Family	How records are produced	Main limitation in small data	No training on target data	Exact size	Columns and types checked	Cost recorded
Interpolation and statistical [4], [16]	Combine real records or sample a simple model fitted to them	Only recombine existing patterns; ignore feature meaning	✓	×	×	×
Deep generative [5], [12], [13]	Train a neural network on the real records and sample from it	Few records to fit many parameters; invalid or copied records	×	✓	×	×
Fine-tuned language models [7], [14]	Fine-tune a language model on records written as text	Training on scarce records	×	✓	×	×
Prompted LLMs [8], [15]	Ask a pre-trained LLM for records, with real examples in the prompt	Unreliable output: wrong size, malformed records, unknown cost	✓	×	×	×
Prompted LLMs, constrained decoding [17], [18]	As above, with the output restricted to a JSON schema	Format guaranteed, but not the number of records; cost unknown	✓	×	✓	×
SYNTHAGENT (ours)	Prompted LLM whose output is checked and completed by a control layer	—	✓	✓	✓	✓

III. THE SYNTHAGENT SYSTEM

SYNTHAGENT meets the constraint of Equation (1)~~by checking the output of the LLMinstead of relying on it: every response is verified, and corrected or completed when necessary, before its records enter the synthetic dataset; the real~~ , a synthetic dataset with exactly N records that conform to S, by construction rather than by trusting the LLM. This guarantee is the core of the system: the LLM only proposes records, and the control layer decides which of them are accepted, asks for corrections or for more records, and stops only when N conforming records have been accepted or a stopping limit is reached. The objectives of Equation (1) are left to the domain knowledge of the LLM and to the real records shown to it, and they are measured, not optimized, in Section VI. The real dataset is never modified. ~~The architecture that implements this principle is shown in~~

Figure 2 shows the architecture and Algorithm 1 the procedure. A profiling step first infers the columns and their types S from the training records. Each request then passes through four stages: the context ~~selector builds the prompt~~ budgeter selects the real records shown to the LLM (1), the LLM proposes a batch of records (2), the ~~parser and validator check~~ deterministic verifier parses, maps and validates it (3), and the ~~accepted~~ admitted records are counted against ~~the configured number~~ N (4). The ~~fixed parameters of these stages~~run produces two outputs: the synthetic table G and an audit trail of every request. The rest of this section follows the procedure in this order; the parameters it uses, and the reason for each value, are ~~listed in Table III; they were set before the experiments and not tuned~~summarized at the end of the section.

A. *Controller State~~of the Control Layer~~*

At every step t, the control layer keeps ~~the~~ an explicit state

$$z_t = (S,\ C_t,\ A_t,\ N - |A_t|,\ B_t,\ E_t), \tag{2}$$

Algorithm 1 ~~Fixed parameters~~ Generation procedure of ~~the control layer and the reason for each value. All were set before the experiments and not tuned; the LLM-chosen settings variant (Section V) lets the LLM choose the first and fourth instead.~~SYNTHAGENT

Require: training records D, columns and types S, number of records N, parameters of Table III

pool $\mathcal{P} \leftarrow$ random sample of $\rho|D|$ records of D
accepted records $A \leftarrow \emptyset$ ▷ empty at the start
repeat
  context $C \leftarrow$ next records of $\mathcal{P}$ that fit in a share ω of the context window W ▷ (1)
  batch $Y \leftarrow$ ask the LLM for $\min(b, N - |A|)$ new records, given S and C ▷ (2)
  records $X \leftarrow$ parse the batch Y, repairing it if needed, and map its columns to S ▷ (3)
  correct the invalid values of X, for at most c rounds ▷ (3)
  $A \leftarrow A \cup X$; log tokens, repairs and violations ▷ (4)
until $|A| \geq N$ **or** $t_{\max}$ rounds **or** $u_{\max}$ empty answers
return synthetic table G, the first N records of A, and the audit trail

~~where~~ made of the columns and types S, the real records C_t ~~is the set of real records~~ currently shown to the LLM, ~~A_t the accepted synthetic records , $N - |A_t|$ the number still missing,~~ the synthetic records A_t accepted so far, the number of records still missing $N - |A_t|$, the number of tokens B_t still available in the context window, and the ~~remaining token budget and E_t the log~~ log E_t of parser and validator results. ~~The~~ From this state the LLM, denoted π_ϕ, proposes a batch $Y_t \sim \pi_\phi(\cdot \mid z_t)$~~. Let~~ , and the accepted set grows only through the verifier:

$$A_{t+1} = A_t \cup K\big(V_S\big(P_S(Y_t)\big)\big), \tag{3}$$

where P_S ~~be the parser, which also maps the returned~~ parses the batch Y_t and maps its columns to S, V_S ~~the validator~~ validates the types, and K ~~a correction step that sends~~ sends the invalid values back to the LLM ~~. The accepted set is updated as Equation (3) expresses the central design decision:~~

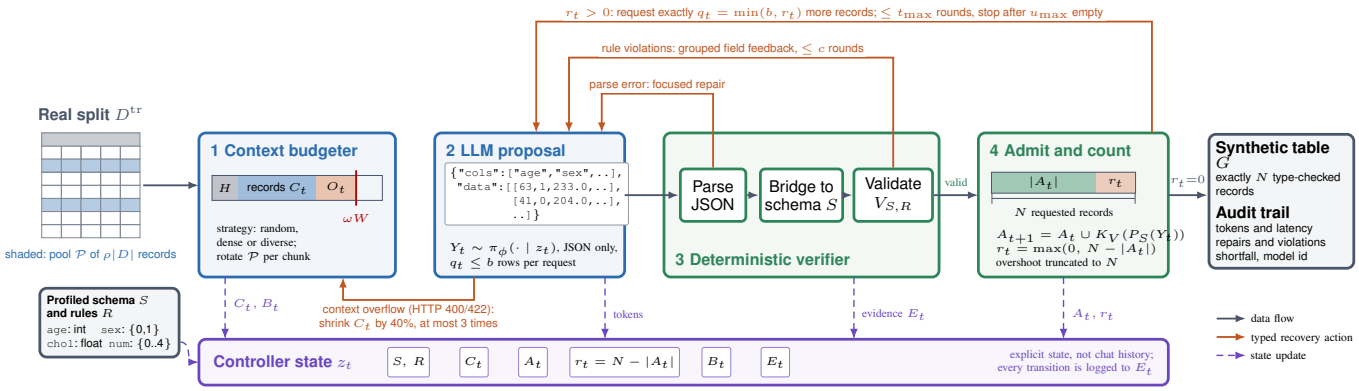


Fig. 2. Architecture of SYNTHAGENT. ~~Real records enter a~~ The context ~~window of limited size~~ budgeter selects the real records shown to the LLM (1); the LLM proposes a ~~JSON~~ batch of records (2); the deterministic ~~checks parse~~ verifier parses it, ~~map~~ maps it to the ~~schema~~ columns and ~~validate the type rules~~ types S and validates it (3); and ~~accepted~~ the admitted records are counted against the ~~requested size~~ configured number N (4). Each failure type triggers its own correction (orange). Every stage reads or updates the controller state z_t ~~of the control layer~~ (purple)~~, which also serves as~~. The outputs are the synthetic table G and the audit ~~log~~ trail of the run.

~~the LLM only proposes, and a record enters A_{t+1} only after it has been parsed, mapped to S and validated~~for correction. Because the state is explicit rather than implied by the conversation history, the number of missing records is always known, and the parser log E_t records every failure for later inspection.

B. Context ~~Selection~~Budgeter (1)

The context ~~selector~~ budgeter decides which real records the LLM sees. ~~Table II lists exactly what the model receives. The core evidence is a sample of $\rho = 20\%$ of the real~~ It draws a random sample of $\rho|D|$ training records, between 31 and 123 records in our datasets, written as compact JSON with the feature columns and the class, so that the model sees how features and class co-occur. ~~No free-text description of the prediction task is added, so that no dataset receives more prompt engineering than another.~~

~~Information boundary for one generation request in the reported study. "Not supplied" is intentional: those signals cannot influence the generated records.~~

~~A selection strategy orders the sample, and the selector shows the model~~ Because a long prompt can exceed the context window W of the LLM, each request shows as many records of the sample as fit in a share ω of the ~~LLM's context window~~W window:

$$C_t = \mathrm{Slice}_t\big(\mathrm{Select}_s(D, \lceil \rho|D| \rceil), k_t\big),$$
$$k_t = \max\{k : T(C_{t,k}) \leq \omega W\}, \tag{4}$$

where Select_s draws the sample with strategy s, T estimates the length in tokens of a prompt ~~that contains~~ with k records~~. The slice~~, and Slice_t rotates through the sample~~across batches, so~~, so that successive requests see different patients. If the provider still rejects a request as too long, the ~~selector~~ budgeter shows fewer records and retries. ~~The default strategy samples at random ;~~ Besides the default random strategy, *dense* prefers records with fewer missing values, and *diverse* selects the records closest to the centres of k-means clusters~~, so that the sample covers more of the feature space~~. Neither strategy is class-stratified, a limitation discussed in Section VII.

C. ~~Parsing, Validation~~ LLM Proposal and ~~Correction~~Prompts (2)

~~Records are requested in batches of $b = 30$. A local parser extracts the recordsfrom each responseand fixes common syntax defects; if~~ The LLM receives the same three-part conversation for every dataset and every LLM (Table II). A first message gives the column names and the sampled records in a compact format, a list of column names followed by one list of values per record. A generation message then asks for an exact number of new records, at most b, and states an output contract: JSON only, the same columns in the same order, a value in every cell, raw numbers, and no additional columns. Repair and correction messages, described next, are added only when a response fails. No free-text description of the dataset or of the prediction task is added, so that no dataset receives more prompt engineering than another. The prompts are therefore deliberately simple, and their wording is not part of the contribution: the guarantees come from the stages that check the response.

D. Deterministic Verifier (3)

The verifier turns a response into records that conform to S, in three steps. *Parsing* extracts the records with a local parser that removes surrounding text and code fences, deletes trailing commas, inserts missing commas between records and closes a response truncated by the output limit. If parsing still fails, the control layer sends one repair request that quotes the position ~~of the error. The~~ and the text around the error, at a low temperature so that the repair fixes the syntax without changing the content. *Mapping* assigns the returned values to the columns of S: by position when the response has the expected number of columns, and otherwise by matching the returned column names ~~are then mapped back to~~ to those of S~~. The validator checks that every value has the type~~ , ignoring

TABLE II

WHAT THE LLM RECEIVES IN ONE GENERATION REQUEST. THE PROMPTS ARE THE SAME FOR EVERY DATASET AND LLM. "NOT SUPPLIED" IS INTENTIONAL: THOSE SIGNALS CANNOT INFLUENCE THE GENERATED RECORDS.

Context layer	What the model receives	Why it is included or excluded
Schema	Ordered column names and an exact column-count requirement	Fixes the record interface and prevents invented or omitted fields.
Demonstrations	Compact row-major serialization of a sample of $\rho\|D\|$ training records, with feature and target values	Exposes joint patterns, value formats, label prevalence and plausible cross-column combinations without sending the whole local table.
Generation task	Exact number of new records for the current batch, at most b	Turns an open-ended completion into a bounded action whose output count can be verified.
Output contract	JSON only, fixed column order, complete cells, raw numeric values, no extra metadata	Makes parsing deterministic and removes interface failures unrelated to statistical quality.
Dynamic feedback	Parser error location after malformed JSON, or grouped field violations after failed validation	Reveals only the evidence needed for a targeted repair instead of replaying the full history.
Free-text metadata	Not supplied	Prevents unequal, dataset-specific prompt engineering from confounding the comparison.
Test information	No holdout rows, predictions, metrics or classifier identities	Preserves the evaluation boundary and prevents optimization against the test set.
Global statistics	No separately computed means, quantiles, correlations or class quotas	Fidelity must arise from the selected records; the consequence is weaker protection against rare-mode omission.

case and spacing; an invented column is dropped and a missing one is left empty. *Validation* checks every value against the type that profiling inferred for its column (integer, real or categorical); optional rules can also bound numerical ranges or restrict categories, but they were off in our experiments ~~.~~ ~~Failures~~ except in the range-enforcement variant. The invalid values are grouped by column and sent back to the LLM with one example ~~of an invalid value~~ per column, for at most ~~$c = 2$ correction rounds~~c correction rounds, so that the model corrects only what is wrong instead of regenerating the batch.

E. ~~*Completion Loop*~~*Admit and Count (4)*

LLMs often return fewer records than requested. ~~After~~ The admitted records are counted against N, and after the planned batches ~~,~~ the completion loop requests only the missing records,

$$r_t = \max(0, N - |A_t|), \qquad q_t = \min(b, r_t), \tag{5}$$

and truncates any excess. It stops when N records have been accepted, after $t_{\max}$ rounds, or after $u_{\max}$ consecutive empty answers; a run that ends short because the provider keeps failing is recorded as such. ~~Algorithm 1 summarizes the complete procedure, with the parameters of Table III. Every run also records the~~ The run returns the synthetic table G and an audit trail with the tokens processed, the generation time, the repairs, the violations and the requested and delivered numbers of records~~; these records make~~, which makes a synthetic dataset inspectable after the fact and ~~give~~ gives the cost κ in Equation (1) a measured value.

~~Generation procedure of~~

F. *Parameters*

Table III lists the parameters of the procedure and the values used in the experiments. They were fixed before the experiments from two practical limits, the size of our datasets and the context and output limits of the LLMs tested,

TABLE III

PARAMETERS OF THE CONTROL LAYER, THE VALUES USED IN THE EXPERIMENTS AND THE REASON FOR EACH VALUE. THE LLM-CHOSEN SETTINGS VARIANT (SECTION V) LETS THE LLM CHOOSE THE FIRST AND FOURTH INSTEAD.

Parameter	Value	Reason
Context sample ρ	20% of training records	31 to 123 records in our datasets: enough to show the main patterns of the data, while the prompt still fits the context window of every LLM tested
Window share ω	85%	Leaves room for the answer and for errors in the estimated prompt length
Batch size b	30 records	The longest answer (30 records of the 30-feature dataset) stays within the output limit of every LLM tested; a failed answer loses at most 30 records
Temperature	0.3	Favours adherence to the output format while keeping some diversity
Repair temperature	0.1	A repair should fix the syntax, not change the content
Correction rounds c	2	Bounds the cost of a batch that keeps failing
Top-up rounds $t_{\max}$	50	$50 \times 30 = 1{,}500$ records, more than twice the largest N (614)
Empty-answer limit $u_{\max}$	5	Five consecutive empty answers indicate a provider failure rather than a sampling accident

rather than tuned on the results, so that no configuration is favoured by tuning on the test data. Their effect is examined in Section VI-A: the LLM-chosen settings variant lets the LLM set the sample size ρ and the temperature, and the context-selection variants change the sample.

G. ~~*Deployment Across Healthcare Facilities*~~

IV. DEPLOYMENT ACROSS HEALTHCARE FACILITIES

As discussed in Section I, healthcare facilities often cannot pool their records [1], [37], and SYNTHAGENT was designed for this setting. Because the control layer needs only a local dataset, its schema and an inference endpoint, each healthcare

facility of a network can run it inside its own infrastructure. Consider K healthcare facilities, each holding a small dataset D_k for the same prediction task, with different case mix, coding practice and class prevalence. Each healthcare facility runs SYNTHAGENT locally and trains on its real and synthetic records,

$$G_k = \mathcal{A}(D_k, S_k, N_k), \qquad \Delta w_k = \mathcal{U}(w_t, D_k \cup G_k), \quad (6)$$

where S_k are the local columns and types, N_k the local number of synthetic records and $\mathcal{U}$ the local training procedure. Only the model update Δw_k and approved aggregate diagnostics leave the healthcare facility. A coordinator aggregates the updates as in FedAvg [38], [39],

$$w_{t+1} = w_t + \sum_{k \in \mathcal{K}_t} \frac{n_k}{\sum_{j \in \mathcal{K}_t} n_j} \Delta w_k, \qquad n_k = |D_k|, \quad (7)$$

preferably under secure aggregation [40]. The weights count *real* records, $|D_k|$ rather than $|D_k \cup G_k|$, so a healthcare facility gains no extra weight by generating more records (Figure 3). Unlike SiloFuse, which trains a shared generator across sites [21], this design needs no generator training across healthcare facilities, and each healthcare facility can reject its synthetic records when paired local validation shows no gain. The design is not, by itself, a privacy guarantee: with a remote LLM the prompt content leaves the healthcare facility, so sensitive deployments need a locally served model or a data-processing agreement, and both synthetic records and model updates can leak information [41]. Our experiments evaluate the generation step one healthcare facility at a time, which is the setting the next section describes; a multi-facility evaluation is future work.

V. EXPERIMENTAL SETUP

This section describes the data, the protocol, the configurations and the measures with which RQ1 to RQ4 are answered.

A. Datasets

Six public classification datasets were selected (Table IV): five from the UCI repository [42]–[46] and the Pima Indians diabetes benchmark [47]. They are widely used, and each has fewer training records than the minimum sample size recommended for developing a prediction model for its task by the criteria of Riley et al. [2]. Together they span the factors that make small data ~~harder~~a challenge for the classifier: 195 to 768 records, 8 to 30 features, two to five classes and a smallest class from 4.3% to 37.3% of the records. We keep the original five-class heart-disease target and impute the few missing values (0.1% of the cells in two datasets) with the mean for numerical and the mode for categorical features.

B. Protocol

Measuring whether synthetic records help a classifier requires real records that the generator has never seen. Each dataset is therefore divided into a training set, given to the generator, and a test set of 20% of the records, used only to score classifiers. Because a test set of 39 to 154 records makes a single division unreliable, the division is repeated ten times with different random seeds, and means and standard deviations are reported. In each repetition, the number of synthetic records N equals the number of training records, so augmentation doubles the training set; because records are requested in batches of at most $b = 30$, a generator that falls short does so on small requests, not because N is large. Ten classifiers covering the main model families are trained: logistic regression (LR), support vector machine (SVM), k-nearest neighbours (kNN), Gaussian naive Bayes (NB), decision tree (DT), random forest (RF), gradient boosting (GB), AdaBoost, XGBoost and a multilayer perceptron (MLP). Using several families avoids conclusions tied to one learner, since interpolated records can help a nearest-neighbour classifier and harm a boosted ensemble. Each classifier is trained in three setups, (1) on real records only, (2) on synthetic records only ~~, and~~ and (3) on real plus synthetic records, and all three are scored on the same test set. XGBoost failed on liver disease in every repetition because ~~the labels are coded~~ it requires classes coded from 0, whereas this dataset codes them as $\{1, 2\}$, so 59 of the 60 classifier–dataset combinations are reported.

TABLE IV
EVALUATION DATASETS. n: RECORDS; d: FEATURES; k: CLASSES; $n_{\min}$: RECORDS IN THE SMALLEST CLASS; $n^\star$: MINIMUM SAMPLE SIZE BY THE FIRST CRITERION OF RILEY ET AL. [2] (SHRINKAGE 0.9, $d(k-1)$ PARAMETERS, ANTICIPATED R^2_{CS} OF 15% OF ITS MAXIMUM); RATIO: $n^\star / n_{\mathrm{tr}}$, WITH $n_{\mathrm{tr}} = 0.8n$ THE TRAINING SIZE.

Dataset	n	d	k	$n_{\min}$ (%)	$n^\star$	Ratio
Heart disease	303	13	5	13 (4.3)	10,201	42.1
Breast cancer	569	30	2	212 (37.3)	2,302	5.1
Heart failure	299	12	2	96 (32.1)	946	4.0
Diabetes	768	8	2	268 (34.9)	621	1.0
Liver disease	583	10	2	167 (28.6)	808	1.7
Parkinson's	195	22	2	48 (24.6)	1,851	11.9

For heart disease, $n^\star$ applies the binary criterion to the smallest class versus the rest and is an approximation. Parkinson's contains several recordings per subject, so its effective number of independent records is lower and its ratio higher than shown.

C. Configurations

Table V lists the 22 configurations. A configuration is included in the analysis only if it completed all six datasets and ten repetitions. The *reference* configuration is SYNTHAGENT with its default settings and GPT-4o mini, the least expensive LLM; every other configuration differs from it in one respect. The *direct-prompting* baseline uses the same LLM, prompts and context, but none of the control layer, and the *structured-output* baseline adds to it the provider's strict JSON-schema mode, which constrains decoding to the column names, numerical types and observed categories of the real data. Eight control-layer variants change one component or repeat the reference run, and three LLM variants replace GPT-4o mini, forming a 2×2 design that crosses model generation (4o versus 4.1) with model size (mini versus full). A second provider, Anthropic, adds three LLMs under the reference configuration, Claude Haiku 4.5, Claude Sonnet 4.6 and Claude Sonnet 5.5, and repeats both baselines with Claude Haiku 5.5, whose

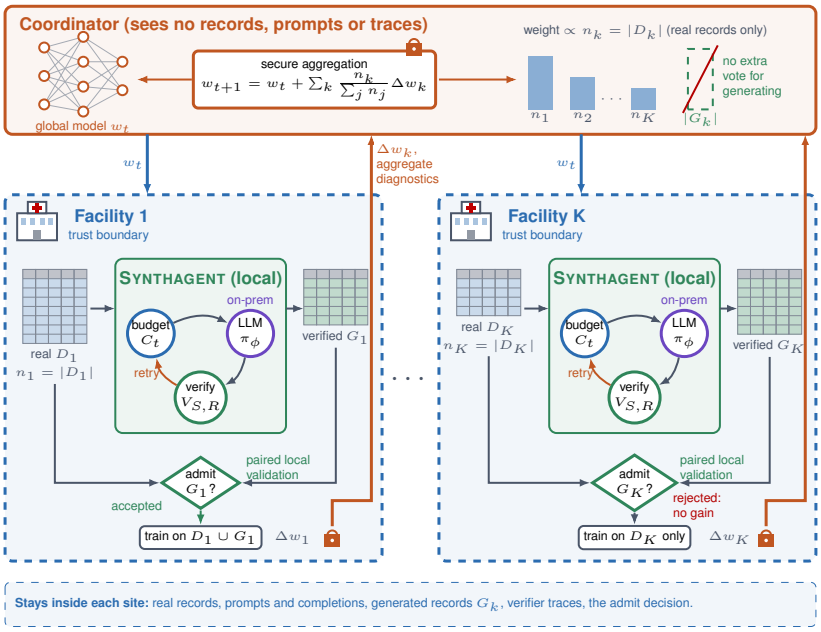


Fig. 3. Deployment across healthcare facilities. Each healthcare facility runs SYNTHAGENT (context selection, a locally served LLM, validation and correction) inside its own infrastructure, then decides with paired local validation whether to use the synthetic records G_k (Facility 1) or to train on real data only (Facility K). Only the model update Δw_k and approved aggregate diagnostics leave the healthcare facility. The coordinator weights each update by the number of real records $n_k = |D_k|$ (Equation (7)), so generating more records gives a healthcare facility no extra weight.

TABLE V
THE 22 EVALUATED CONFIGURATIONS. ALL LLM CONFIGURATIONS USE GPT-4O MINI, EXCEPT THE LLM VARIANTS AND THE BASELINES REPEATED WITH CLAUDE HAIKU 5.5.

Group	Configurations	No.	Purpose
Baselines (RQ1)	Direct prompting; structured outputs, each with GPT-4o mini and Claude Haiku 5.5	4	Same LLM without the control layer, with free or schema-constrained decoding
Reference	SYNTHAGENT, default settings	1	Point of comparison for all others
Repeated runs (RQ1)	No type rules; no post-processing; no context join; logged rerun	4	Change nothing in this setting; estimate the variation between runs
Design choices (RQ1)	Dense context; diverse context; LLM-chosen settings; range enforcement	4	Effect of context selection, of letting the LLM set ρ and the temperature, and of clipping values to the observed ranges
LLM variants (RQ4)	GPT-4.1 mini; GPT-4o; GPT-4.1; Claude Haiku 4.5; Claude Sonnet 4.6; Claude Sonnet 5.5	6	Effect of model generation, size and provider
Established generators (RQ4)	SMOTE; CTGAN; TVAE	3	Generators fitted to the real records
Total		22	

reference run is reported separately in Section VI-A. The Claude models run without extended reasoning and at their default temperature, which the newest of them do not allow to be changed, whereas the OpenAI models use the temperature of Table III. Token counts are compared only within a provider, because the tokenizers differ. Three established generators, SMOTE and CTGAN and TVAE with the default settings of their reference implementations, use the same divisions. In total, the configurations give 1,320 dataset–configuration runs and up to 39,600 classifier fits.

Three of the control-layer variants change nothing in this setting: with no user rules, disabling post-processing removes a step that does nothing, the context-join option affects only a legacy fidelity score, and disabling type rules cannot change the output because the validator never found a type violation with GPT-4o mini. Together with the reference and a rerun of it that only adds per-request logging, they are five independent runs of the same pipeline, and their spread estimates the variation between repeated runs, which we call the *noise floor*: a difference between two configurations that is smaller than this spread is not evidence of an effect.

D. Measures

Each research question is answered with its own measures, all computed on every repetition: reliability for RQ1, fidelity and privacy for RQ2, and utility for RQ3 and RQ4, with cost reported throughout. *Reliability* is measured by the number of records delivered against the number configured, the number of malformed responses, the percentage of non-empty values, the corrections made and the cost in tokens. *Fidelity* compares the synthetic records alone with the real training records over the numerical features $\mathcal{J}$:

$$F_{\mathrm{JSD}} = 100\Big(1 - \tfrac{1}{|\mathcal{J}|}\sum_{j\in\mathcal{J}} D_{\mathrm{JS}}(D_j, G_j)\Big),$$
$$F_{\mathrm{corr}} = 100\big(1 - \mathrm{mean}\,|\Sigma_D - \Sigma_G|\big), \tag{8}$$

where D_{JS} is the Jensen–Shannon divergence between real and synthetic distributions of a feature and Σ_D, Σ_G are the correlation matrices; both scores equal 100 for identical distributions. A classifier two-sample test (C2ST) [32] trains a random forest to separate real from synthetic records; an accuracy close to 0.5 means that they are hard to tell apart. For the established generators and direct prompting we also report the percentage of *valid* records, whose values lie in the observed ranges and categories and are integers where

the real feature is, a stricter criterion than the column-and-type checks of the control layer. *Privacy* is measured by a membership-inference attack that scores each real record by its distance to the nearest synthetic record and reports the ROC AUC between training and test records, where 0.5 is chance. *Utility* is the paired change, for metric $m \in \{\text{accuracy}, \text{F1}\}$, dataset d, classifier c and repetition h,

$$\Delta m_{d,c,h} = m\big(D_{d,h} \cup G_{d,h};\, c\big) - m\big(D_{d,h};\, c\big), \qquad (9)$$

with both models scored on the same test set. We use binary F1 for the binary tasks and class-weighted F1 for heart disease, and report changes in percentage points (pp).

E. Statistical Analysis

Repetitions and classifiers are not independent datasets, so they are averaged within each dataset and the six datasets are treated as matched blocks. Configurations are compared with the Friedman test,

$$\chi_F^2 = \frac{12M}{Q(Q+1)} \sum_{a=1}^{Q} \bar{\text{rk}}_a^{\,2} - 3M(Q+1), \qquad (10)$$

where $\bar{\text{rk}}_a$ is the average rank of configuration a over $M = 6$ datasets and Q the number of configurations, followed by two-sided Wilcoxon signed-rank tests of the best-ranked configuration against every other, with Holm correction [48], [49]. With six blocks the tests have limited power, so effect sizes and the noise floor are reported alongside p-values. As a sensitivity analysis, the comparison of LLMs is repeated with the 60 dataset–division pairs as blocks: all configurations use the same divisions, so the pairs are matched, but divisions of one dataset share records, so these blocks are not independent and the dataset-level tests remain the primary analysis.

VI. Results

A. RQ1: Control Layer and Reliability

The control layer and the reliability of the generated datasets are directly related: without it, the same LLM never delivered the configured number of records; with it, every dataset was complete, at 1.9% more tokens and with unchanged quality. Table VI compares direct prompting with the reference SYNTHAGENT on the same prompts, context and divisions. Direct prompting delivered on average 87.6% of the configured records, fell short in 50 of 60 runs and overshot in the other 10, and lost an entire batch to each of ten malformed responses. SYNTHAGENT delivers exactly N records because its completion loop requests the missing ones and truncates any excess, and its parser recovers malformed responses instead of discarding them. Fidelity, privacy and utility were the same for both configurations (JSD 72.7; attack AUC 0.517 versus 0.526; F1 change -0.04 versus -0.14 pp, within the noise floor below), so the difference lies not in the records but in whether the dataset that was configured is delivered.

Structured outputs, the option offered by LLM providers to force every response into a fixed format, do not replace the control layer (Table VI). Every response had the declared columns and types, but the LLM returned on average 77.7% of the configured records and never the full dataset, at 21% more tokens than SYNTHAGENT. Forcing the format also weakened the relationships between features (correlation fidelity 63.0 versus 72.6) and made the records easier to tell from real ones (C2ST 0.943 versus 0.878), while utility did not differ (Friedman $p = .311$ for F1). A fixed format therefore guarantees the shape of each record, but not the size of the dataset, which is what the completion loop provides. The same holds for the second provider: with Claude Haiku 5.5, direct prompting delivered on average over datasets 67.1% of the configured records, only 10.5% on breast cancer, with 294 malformed responses, and structured outputs delivered 82.9%, exceeding the configured size on four datasets and reaching only 37.6% and 42.1% on breast cancer and Parkinson's. Under the control layer, the same model declined in plain text to imitate the patient records in 39 of the 60 repetitions of a first pass; SYNTHAGENT logged each refusal instead of returning an incomplete dataset, but completed only heart failure, so this run is not included in the comparisons.

TABLE VI
DIRECT PROMPTING, STRUCTURED OUTPUTS AND SYNTHAGENT (RQ1), ALL WITH GPT-4O MINI, OVER TEN DIVISIONS. DELIV.: RECORDS RETURNED AS A PERCENTAGE OF THE CONFIGURED NUMBER N (RANGE OVER DIVISIONS FOR DIRECT PROMPTING); MALF.: MALFORMED RESPONSES; ΔF1: MEAN F1 CHANGE (PP). SYNTHAGENT DELIVERED 100% IN EVERY RUN.

		Direct prompting			Structured		SYNTHAGENT
Dataset	N	Deliv. (%)	Malf.	ΔF1	Deliv.	ΔF1	ΔF1
Heart disease	242	93.5 (91.7–94.6)	0	-0.15	82.1	-0.88	-0.97
Breast cancer	455	64.0 (56.9–69.0)	8	-0.78	63.9	-0.97	-0.62
Heart failure	239	94.5 (93.3–95.8)	0	$+0.26$	79.3	$+2.65$	-1.00
Diabetes	614	102.2 (101.3–103.4)	0	$+2.23$	93.2	$+2.90$	$+2.39$
Liver disease	466	93.8 (92.7–95.5)	0	$+0.12$	76.8	-0.71	$+0.13$
Parkinson's	156	77.3 (62.2–87.8)	2	-1.94	70.8	-0.73	-0.77
Mean		87.6	10	-0.04	77.7	$+0.38$	-0.14

The design choices inside the control layer matter much less than the control layer itself (Table VII). The five repeated runs of the same pipeline differ by up to 0.5 JSD points, 2 correlation points and 0.7 pp of utility, which is the noise floor. Dense context stays inside it; diverse context slightly improves fidelity (JSD 73.27, correlation 73.72) at the cost of a slightly higher attack AUC (0.544); letting the LLM choose its own settings changes nothing but costs 57% more tokens; and clipping values to the observed ranges lowers JSD by 0.9 points, just beyond the noise floor, without improving utility. The Friedman tests find no difference in utility between the variants ($p = .843$ for accuracy, $p = .872$ for F1; Figure 4). Generation time varied by a factor of 1.7 between the repeated runs, depending on the provider, while their token counts differed by less than 2%, so cost is measured in tokens.

The corrections are what allow a less disciplined LLM to be used. GPT-4o mini produced no type violation, and the local parser fixed every defect in its responses. GPT-4.1 mini, the fastest LLM, produced on average 7.4 type violations per repetition on breast cancer and one malformed response; these

TABLE VII

CONTROL-LAYER VARIANTS (RQ1), ALL WITH GPT-4O MINI. THE FIRST FIVE ROWS RUN THE SAME PIPELINE AND DEFINE THE NOISE FLOOR. SECONDS AND TOKENS ARE MEANS PER REPETITION. BOLD: BEST VALUE AMONG THE LLM CONFIGURATIONS; LOWER IS BETTER FOR C2ST, FOR THE DISTANCE OF THE ATTACK AUC FROM 0.5, AND FOR COST.

	Fidelity and privacy				Utility change (pp)		Cost per repetition	
Configuration	JSD	Corr.	C2ST	Attack AUC	Acc.	F1	Seconds	Tokens
Noise floor: five runs of the same pipeline								
Reference SYNTHAGENT	72.70	72.57	0.878	0.526	−1.19	−0.14	378.7	73,510
No type rules	72.92	72.93	0.876	0.534	−1.80	−0.04	321.6	72,188
No post-processing	72.43	70.83	0.883	**0.523**	−1.71	+0.28	251.5	72,235
No context join	72.52	71.18	0.880	0.531	−1.31	−0.18	**228.1**	72,576
Logged rerun	72.72	72.05	0.885	0.530	−1.32	−0.45	282.9	73,235
Design choices								
Dense context	72.87	70.88	0.878	0.525	−1.24	+0.07	238.8	72,600
Diverse context	**73.27**	**73.72**	0.872	0.544	−1.34	−0.13	271.1	73,757
LLM-chosen settings	72.53	70.10	0.886	0.526	−1.25	+0.11	310.2	115,245
Range enforcement	71.83	71.72	**0.870**	0.534	−1.13	−0.79	339.8	**71,915**

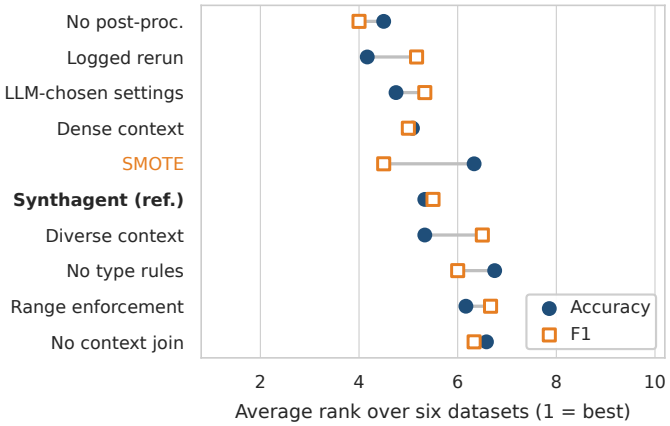


Fig. 4. Average ranks of the control-layer variants and SMOTE over the six datasets (lower is better). Neither accuracy nor F1 shows a significant difference.

were sent back for correction instead of entering the dataset, and its datasets were complete and gave the best accuracy change of all configurations (Table X).

B. RQ2: Fidelity and Privacy

The generated records follow the real distributions reasonably well without revealing the real records, but they remain distinguishable from them (Table VIII). Fidelity averages 72.7 out of 100, from 65.2 on the five-class heart-disease dataset, the furthest below its required sample size, to 80.2 on diabetes, the closest to it. The membership-inference attack stays near chance (AUC 0.526 on average, at most 0.567), which is a diagnostic for one attack rather than a formal guarantee. The C2ST accuracy of 0.878 shows that a classifier can still separate real from synthetic records, so the synthetic records complement real ones rather than replace them.

C. RQ3: Utility for Classifiers Trained on Small Data

The generated records help the classifiers that fail on the minority class and slightly harm those that are already strong, so their effect depends on the dataset and the classifier.

TABLE VIII

REFERENCE CONFIGURATION (GPT-4O MINI) PER DATASET, AVERAGED OVER TEN REPETITIONS AND ALL CLASSIFIERS (RQ2, RQ3). JSD AND CORRELATION ARE ON A 0–100 SCALE.

	Fidelity and privacy				Acc. (%)		Change (pp)	
Dataset	JSD	Corr.	C2ST	AUC	Real	Aug.	Acc.	F1
Breast	77.2	74.5	0.911	0.550	95.1	94.4	−0.66	−0.62
Diabetes	80.2	44.0	0.897	0.504	74.2	74.1	−0.03	+2.39
Heart dis.	65.2	76.6	0.893	0.507	56.6	53.4	−3.15	−0.97
Heart fail.	71.2	81.2	0.892	0.534	74.1	71.6	−2.53	−1.00
Liver	71.8	73.8	0.883	0.495	67.4	67.6	+0.19	+0.13
Parkinson's	70.6	85.3	0.789	0.567	84.9	84.0	−0.97	−0.77
Mean	72.7	72.6	0.878	0.526	75.4	74.2	−1.19	−0.14

Classifiers trained on real data alone already vary widely (Table IX): on the 195-record Parkinson's dataset, MLP accuracy is $72.3 \pm 18.2\%$ while tree ensembles exceed 92%, and on heart failure SVM and kNN reach over 60% accuracy with an F1 of almost zero because they rarely predict the minority class. With the reference configuration, F1 improves in 26 of the 59 classifier–dataset combinations and decreases in 31 (Figure 5); the largest gains occur where real-only training fails, such as kNN on heart failure (+15.3 F1 points) and MLP on Parkinson's (+8.2 accuracy points). No single combination is significant after Holm correction, and Figure 6 shows the variation between divisions. On average, F1 is unchanged with GPT-4o mini (−0.14 pp) and increases by 1.09 pp with GPT-4.1, which is consistent with the bound set by information theory: synthetic records cannot carry more information about the task than the real records and the generator's knowledge together ~~[6]~~ [6, Sec. 2.8].

D. RQ4: Comparison with Established Generators and LLMs

SYNTHAGENT gives the largest and most stable F1 gain among the generators compared, and the LLM is the only factor whose effect exceeds the noise floor. Among the LLMs (Table X, Figure 7), the Claude models give the strongest results in our runs: Claude Sonnet 4.6 reaches the highest

TABLE IX

REAL-ONLY BASELINES (RQ3) OVER TEN 80/20 DIVISIONS: MEAN IN PERCENT, STANDARD DEVIATION IN SMALL TYPE. XGBOOST ON LIVER DISEASE (–) FAILED IN EVERY DIVISION BECAUSE THE LABELS ARE CODED $\{1, 2\}$.

	Heart dis.		Breast		Heart fail.		Diabetes		Liver		Parkinson's	
Classifier	Acc.	F1	Acc.	F1	Acc.	F1	Acc.	F1	Acc.	F1	Acc.	F1
RF	61.0 ±4.4	57.0 ±5.9	95.6 ±1.8	95.0 ±2.8	84.3 ±3.3	76.8 ±4.4	74.9 ±2.8	60.4 ±6.0	71.1 ±3.5	81.4 ±2.4	92.3 ±3.2	94.8 ±2.2
DT	53.4 ±3.9	54.0 ±5.0	93.3 ±2.3	92.1 ±4.0	79.2 ±4.7	68.4 ±9.2	71.1 ±3.8	57.2 ±6.4	66.0 ±3.2	76.3 ±3.1	85.1 ±5.6	89.7 ±3.8
MLP	56.6 ±6.4	52.5 ±7.6	94.7 ±0.9	94.0 ±2.1	48.3 ±12.9	34.2 ±24.7	68.4 ±4.1	52.9 ±12.0	68.4 ±4.1	79.4 ±3.6	72.3 ±18.2	77.8 ±26.1
SVM	54.8 ±6.8	39.0 ±7.9	93.3 ±2.1	92.4 ±3.4	63.3 ±5.2	0.0 ±0.0	75.6 ±2.2	57.3 ±4.7	70.7 ±2.9	82.8 ±2.0	81.0 ±8.2	88.5 ±5.2
LR	59.8 ±5.6	56.5 ±5.0	95.2 ±2.0	94.6 ±2.6	80.5 ±3.6	68.9 ±8.4	77.3 ±2.0	62.8 ±5.4	70.3 ±4.0	81.1 ±3.2	86.9 ±5.6	91.3 ±3.9
kNN	55.4 ±6.3	44.4 ±7.9	94.3 ±1.8	93.4 ±3.2	61.3 ±4.1	0.7 ±2.1	75.5 ±1.8	60.0 ±3.3	68.6 ±4.2	79.7 ±3.2	83.6 ±5.6	89.5 ±3.9
NB	55.7 ±5.3	56.9 ±5.0	93.9 ±1.7	93.1 ±3.2	75.0 ±4.9	56.7 ±13.1	76.4 ±2.4	64.0 ±4.9	55.9 ±3.7	55.9 ±3.8	71.5 ±4.4	76.6 ±4.0
GB	54.1 ±5.5	52.8 ±6.3	95.9 ±2.2	95.4 ±2.8	83.0 ±4.9	75.4 ±7.2	74.7 ±3.0	61.7 ±5.8	67.9 ±3.3	78.7 ±2.4	92.1 ±4.0	94.7 ±2.6
AdaBoost	60.8 ±7.6	60.7 ±7.3	97.1 ±1.5	96.8 ±1.7	82.5 ±3.0	74.2 ±6.5	74.9 ±2.2	60.8 ±4.7	67.4 ±3.4	78.2 ±3.3	91.3 ±3.8	94.2 ±2.5
XGBoost	54.3 ±4.6	52.0 ±5.1	97.2 ±1.4	96.9 ±1.8	83.5 ±4.0	75.5 ±7.0	72.7 ±2.8	59.1 ±6.2	–	–	93.3 ±5.2	95.6 ±3.2

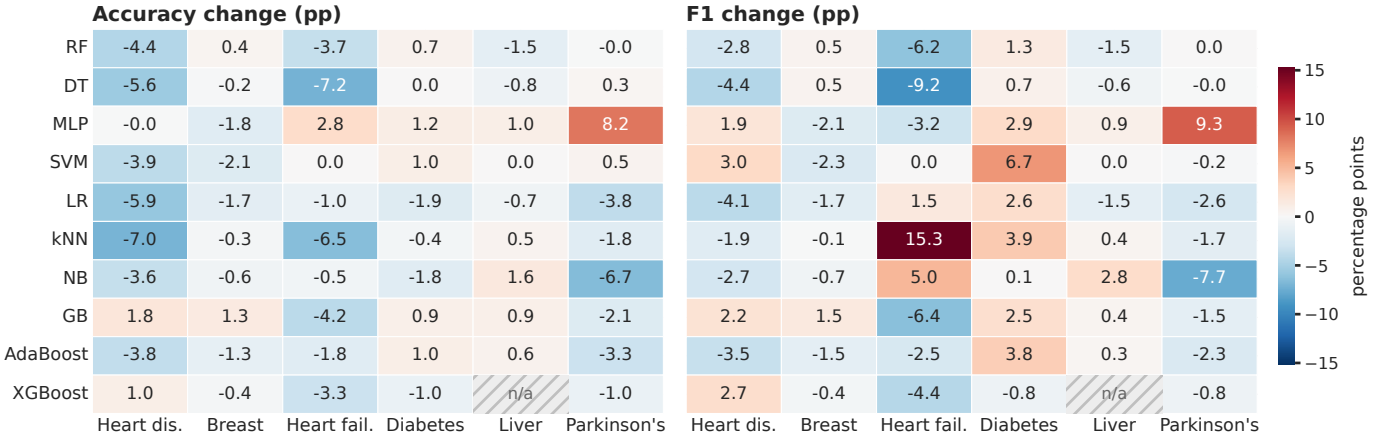


Fig. 5. Change from real-only to real-plus-synthetic training for the reference configuration (RQ3), per classifier and dataset. Hatched cells (n/a) were not evaluated. The same synthetic data can help one classifier and harm another.

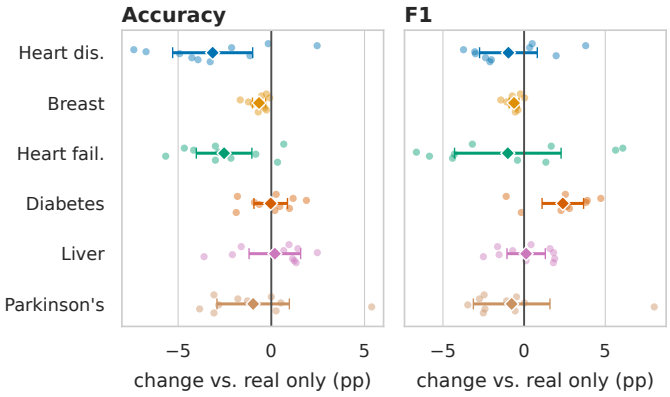


Fig. 6. Variation between divisions for the reference configuration (RQ3). Points are divisions, after averaging the paired changes over classifiers; diamonds are means and bars 95% t intervals.

fidelity (79.07) and the largest gains (+0.40 pp of accuracy, +2.39 pp of F1), and all three Claude models exceed the best OpenAI model in F1. This comparison between providers is suggestive rather than conclusive: the Claude models ran at their default temperature and the OpenAI models at ~~0.1~~0.3, and more varied sampling may account for part of the difference. Within OpenAI, GPT-4.1 gives the highest fidelity (75.72) and F1 gain (+1.09 pp) and GPT-4.1 mini the best accuracy change at the lowest generation time, while the three stronger OpenAI models preserve correlations between features less well (62.8 to 64.3, against 72.6 for GPT-4o mini). The control layer completed every dataset with every one of these LLMs, and the differences between them are not significant across six datasets ($p = .572$ and $p = .615$ within OpenAI, $p = .377$ over all seven, for accuracy and F1). With the 60 divisions as blocks, the OpenAI models still do not differ ($p = .131$ and $p = .235$), but the seven LLMs do ($p = .002$ for accuracy, $p = .001$ for F1): Claude Sonnet 4.6 gives a higher F1 than each of the other six LLMs after Holm correction ($p \leq .039$) and a higher accuracy than GPT-4o mini, GPT-4o and GPT-4.1 ($p \leq .017$).

Against the established generators (Table XI), no method is best on every measure. CTGAN reaches the highest fidelity (77.6), but its records are the easiest to detect (C2ST 0.936) and only 29.6% of them are valid; TVAE keeps 75.6% valid records and SMOTE 85.7%, but only 14.4% on heart disease, where interpolation produces fractional values in integer-coded features. All keep the membership attack near chance. The established generators are much cheaper, needing seconds of

TABLE X

LLMs UNDER THE REFERENCE CONFIGURATION (RQ4). SECONDS AND TOKENS ARE MEANS PER REPETITION; TOKENS ARE COMPARABLE ONLY WITHIN A PROVIDER. ALL DATASETS WERE COMPLETE.

LLM	JSD	Corr.	ΔAcc.	ΔF1	Sec.	Tokens
GPT-4o mini	72.70	**72.57**	−1.19	−0.14	378.7	73,510
GPT-4.1 mini	73.60	62.82	−0.23	+0.92	**168.5**	**72,069**
GPT-4o	73.95	64.32	−0.50	+0.25	255.2	80,046
GPT-4.1	75.72	63.30	−0.44	+1.09	201.7	80,652
Claude Haiku 4.5	74.85	69.88	−0.34	+1.55	253.5	78,020
Claude Sonnet 4.6	**79.07**	70.77	+0.40	+2.39	373.5	**75,504**
Claude Sonnet 5.5	79.00	63.78	+0.10	+1.62	205.5	86,074

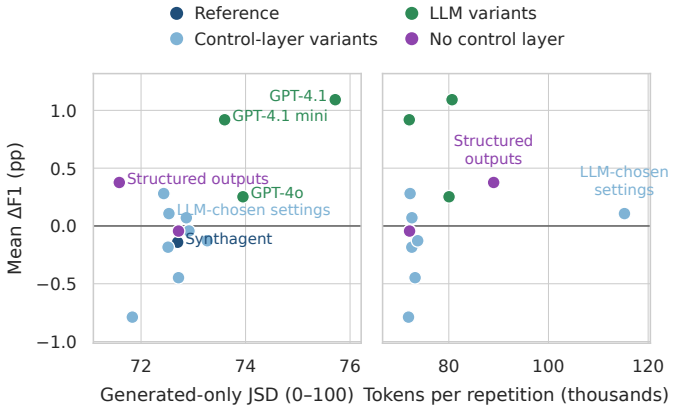


Fig. 7. Fidelity, F1 change and token cost of the OpenAI configurations (RQ1, RQ4); the Claude models are omitted because their token counts are not comparable. Control-layer variants cluster inside the noise floor; the LLMs spread along fidelity and F1; without the control layer, structured outputs cost the most tokens of the GPT-4o mini configurations except LLM-chosen settings.

CPU time instead of about 80,000 tokens.

The decisive difference appears per dataset (Table XII). Synthetic records can harm a classifier when they misrepresent the real data, and in small data a large loss on one dataset matters more than a small average gain. SMOTE gains 10.91 pp of F1 on heart failure but loses 8.20 pp on liver disease; CTGAN loses on every dataset, by up to 10.65 pp; TVAE loses 2.86 pp on heart failure. SYNTHAGENT with GPT-4.1 combines a positive average gain (+1.09 pp) with the smallest worst case (−0.53 pp), while gaining 6.35 pp on heart failure, and with Claude Sonnet 4.6 the average gain rises to +2.39 pp with a worst case of −0.61 pp. Across datasets the differences between the five generators of Table XI are not significant ($p = .147$ for accuracy, $p = .231$ for F1), so TVAE remains a cheap option for accuracy and SMOTE for a known rare positive class.

VII. DISCUSSION AND LIMITATIONS

The objective of this work was to determine whether SYNTHAGENT makes LLM-based SDG a reliable and useful option for small healthcare datasets. Reliability is achieved, and it is achieved by the control layer: the same LLM, prompted directly, never delivered the configured dataset,

TABLE XI

GENERATORS COMPARED ON THE SAME DIVISIONS (RQ4). VALID: % OF RECORDS WITHIN THE OBSERVED RANGES, CATEGORIES AND INTEGER TYPES (NOT LOGGED FOR THE LLM RUNS, WHOSE TYPES ARE ENFORCED BY THE VALIDATOR). SECONDS: API TIME FOR SYNTHAGENT, CPU TIME OTHERWISE.

Generator	JSD	Corr.	C2ST	AUC	Valid	ΔAcc.	ΔF1	Sec.	Tok.
SYNTHAGENT, 4o mini	72.7	72.6	0.878	0.526	–	−1.19	−0.14	379	73.5k
SYNTHAGENT, 4.1	75.7	63.3	0.877	0.527	–	−0.44	+1.09	202	80.7k
SMOTE†	69.9	85.0	0.795	0.532	**85.7**	−3.08	+0.33	**0.01**	0
CTGAN	**77.6**	75.2	0.936	**0.503**	29.6	−1.40	−3.37	14.6	0
TVAE	71.9	**85.6**	0.868	0.515	75.6	−0.09	−0.21	5.6	0

†Emits only non-majority records, so its C2ST is not comparable.

TABLE XII

MEAN F1 CHANGE PER DATASET (PP) OVER TEN DIVISIONS AND ALL CLASSIFIERS (RQ4). SA: SYNTHAGENT. WORST: LARGEST LOSS ON ANY DATASET.

Generator	Heart d.	Breast	Heart f.	Diab.	Liver	Park.	Mean	Worst
SA, 4o mini	−0.97	−0.62	−1.00	+2.39	+0.13	−0.77	−0.14	−1.00
SA, 4.1	−0.44	−0.07	+6.35	+1.75	−0.50	−0.53	+1.09	−0.53
SA, Son. 4.6	+3.74	−0.36	+8.69	+3.22	−0.61	−0.37	+2.39	−0.61
SMOTE	−1.50	−0.53	+10.91	+4.11	−8.20	−2.83	+0.33	−8.20
CTGAN	−2.30	−2.00	−10.65	−4.22	−0.65	−0.38	−3.37	−10.65
TVAE	−0.73	−0.88	−2.86	+0.89	+2.21	+0.10	−0.21	−2.86

whereas SYNTHAGENT completed it, recorded its cost and caught the failures of a less disciplined model at almost no extra cost (RQ1). Usefulness is real but bounded: the synthetic records ~~are faithful~~ preserve the distribution of the real data without revealing real ~~ones~~ records (RQ2), they help the classifiers that fail on the minority class (RQ3), and with a stronger LLM they give the largest and most stable gain among the generators compared (RQ4). No generator, however, improves every dataset, which is consistent with the data-processing inequality ~~[6]~~ [6, Sec. 2.8]: SDG cannot replace missing patients, and its benefit is bounded by the knowledge the generator brings. Among the generators compared, only the LLM brings such knowledge, and the control layer is what makes it usable. In practice, the choice of LLM matters most, cost is better measured in tokens than in time, and the effect of synthetic records should be checked per classifier before they are used.

The conclusions are subject to several limitations. Only six datasets act as independent blocks, which limits the power of the tests, and the divisions are not stratified. The datasets are public and widely used, so they may be part of the LLMs' training data and part of the fidelity may reflect memorized records rather than generation; the membership-inference attack does not test for this, and an evaluation on private data is left for future work. The Parkinson's dataset contains several recordings per subject, so recordings of one person can appear in both sets. The columns and types are inferred from the full dataset rather than from the training set alone. The parameters in Table III were fixed and not tuned, and only two of them were varied, through the LLM-chosen settings variant. SYNTHAGENT does not filter synthetic records that are close to real ones: privacy is measured

with one distance-based attack, not enforced, and no result establishes differential privacy [36]. The SYNTHAGENT runs predate the counter of delivered records, so their size follows from the completion loop rather than from a logged count, and the validity measure was logged only for the established generators and direct prompting (78.6% valid records). The corrections were exercised only by GPT-4.1 mini, and records obtained in the last correction round or in the completion loop are not validated again. Context selection is not class-stratified, so a rare class can be missing from a prompt. The LLMs come from two commercial providers and locally served models were not evaluated; one of them, Claude Haiku 5.5, declined to imitate patient records under the control layer's prompts, which shows that a provider's safety behaviour can block generation on healthcare data. CTGAN and TVAE use default settings.

Synthetic healthcare records can support development, teaching and software testing, but they must not inform diagnosis or policy without domain review, and near-chance performance against one attack does not justify releasing sensitive data. The implementation, the experiment runner and the script that regenerates every table and figure are publicly available.[1]

VIII. CONCLUSION

Small healthcare datasets pose a dilemma for synthetic data generation: generators fitted to the available records can only recombine the patterns those records contain, whereas an LLM can add domain knowledge but returns text rather than a dataset. SYNTHAGENT resolves the dilemma with a control layer that turns the responses of an LLM into complete synthetic datasets with the columns and types of the real data and a recorded cost, without training on the target data. On six public healthcare datasets, direct prompting never delivered the configured dataset, with LLMs of two providers and even with responses forced into a fixed format, while SYNTHAGENT did at 1.9% more cost, and with the strongest LLM it gave the largest and most stable F1 gain of five generators. Because repeated runs of the same pipeline vary as much as most design choices, we recommend that SDG for small data be evaluated on repeated runs. Future work will evaluate locally served LLMs, class-aware context selection, privacy filters and the deployment across healthcare facilities outlined in Section IV.

ACKNOWLEDGMENT

This work is funded by national funds through FCT – Foundation for Science and Technology, I.P., within the scope of the research unit UID/00326 – Centre for Informatics and Systems of the University of Coimbra, https://doi.org/10.54499/UID/00326/2025.

AI-GENERATED CONTENT ACKNOWLEDGEMENT

The authors used Claude (Anthropic) to assist with code development and with the writing of the manuscript, and take full responsibility for all content.

[1]TODO(authors): add the public repository URL required by the ICDE artifact policy.

REFERENCES

[1] N. Rieke, J. Hancox, W. Li, F. Milletar'i, H. R. Roth, S. Albarqouni, S. Bakas, M. N. Galtier, B. A. Landman, K. Maier-Hein, S. Ourselin, M. Sheller, R. M. Summers, A. Trask, D. Xu, M. Baust, and M. J. Cardoso, "The future of digital health with federated learning," *npj Digital Medicine*, vol. 3, no. 1, p. 119, 2020.

[2] R. D. Riley, K. I. E. Snell, J. Ensor, D. L. Burke, F. E. Harrell, K. G. M. Moons, and G. S. Collins, "Minimum sample size for developing a multivariable prediction model: Part II – binary and time-to-event outcomes," *Statistics in Medicine*, vol. 38, no. 7, pp. 1276–1296, 2019.

[3] Z. Qian, R. Davis, and M. van der Schaar, "Synthcity: A benchmark framework for diverse use cases of tabular synthetic data," in *Advances in Neural Information Processing Systems*, vol. 36, 2023.

[4] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic minority over-sampling technique," *Journal of Artificial Intelligence Research*, vol. 16, pp. 321–357, 2002.

[5] L. Xu, M. Skoularidou, A. Cuesta-Infante, and K. Veeramachaneni, "Modeling tabular data using conditional GAN," in *Advances in Neural Information Processing Systems*, vol. 32, 2019.

[6] T. M. Cover and J. A. Thomas, *Elements of Information Theory*, 2nd ed. Hoboken, NJ: Wiley-Interscience, 2006.

[7] V. Borisov, K. Sessler, T. Leemann, M. Pawelczyk, and G. Kasneci, "Language models are realistic tabular data generators," in *International Conference on Learning Representations*, 2023. [Online]. Available: https://openreview.net/forum?id=cEygmQNOeI

[8] N. Seedat, N. Huynh, B. van Breugel, and M. van der Schaar, "Curated LLM: Synergy of LLMs and data curation for tabular augmentation in low-data regimes," in *Proceedings of the 41st International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 235, 2024.

[9] R. Y. Wang and D. M. Strong, "Beyond accuracy: What data quality means to data consumers," *Journal of Management Information Systems*, vol. 12, no. 4, pp. 5–33, 1996.

[10] C. Batini, C. Cappiello, C. Francalanci, and A. Maurino, "Methodologies for data quality assessment and improvement," *ACM Computing Surveys*, vol. 41, no. 3, pp. 16:1–16:52, 2009.

[11] A. D. Lautrup, T. Hyrup, A. Zimek, and P. Schneider-Kamp, "SynthEval: A framework for detailed utility and privacy evaluation of tabular synthetic data," *Data Mining and Knowledge Discovery*, vol. 39, no. 1, p. 6, 2025.

[12] A. Kotelnikov, D. Baranchuk, I. Rubachev, and A. Babenko, "TabDDPM: Modelling tabular data with diffusion models," in *Proceedings of the 40th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 202, 2023, pp. 17 564–17 579.

[13] H. Zhang, J. Zhang, Z. Shen, B. Srinivasan, X. Qin, C. Faloutsos, H. Rangwala, and G. Karypis, "Mixed-type tabular data synthesis with score-based diffusion in latent space," in *International Conference on Learning Representations*, 2024. [Online]. Available: https://openreview.net/forum?id=4Ay23yeuz0

[14] A. V. Solatorio and O. Dupriez, "REaLTabFormer: Generating realistic relational and tabular data using transformers," *arXiv preprint arXiv:2302.02041*, 2023.

[15] J. Kim, T. Kim, and J. Choo, "EPIC: Effective prompting for imbalanced-class data synthesis in tabular data classification via large language models," in *Advances in Neural Information Processing Systems*, vol. 37, 2024.

[16] J. Zhang, G. Cormode, C. M. Procopiuc, D. Srivastava, and X. Xiao, "PrivBayes: Private data release via Bayesian networks," in *Proceedings of the ACM SIGMOD International Conference on Management of Data*, 2014.

[17] B. T. Willard and R. Louf, "Efficient guided generation for large language models," *arXiv preprint arXiv:2307.09702*, 2023.

[18] S. Geng, M. Josifoski, M. Peyrard, and R. West, "Grammar-constrained decoding for structured NLP tasks without finetuning," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023, pp. 10 932–10 952.

[19] C. Lee, J. Kim, and N. Park, "CoDi: Co-evolving contrastive diffusion models for mixed-type tabular synthesis," in *Proceedings of the 40th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 202, 2023, pp. 18 940–18 956.

[20] J. Fan, T. Liu, G. Li, J. Chen, Y. Shen, and X. Du, "Relational data synthesis using generative adversarial networks: A design space

exploration," *Proceedings of the VLDB Endowment*, vol. 13, no. 11, pp. 1962–1975, 2020.

[21] A. Shankar, H. Brouwer, R. Hai, and L. Y. Chen, "SiloFuse: Cross-silo synthetic data generation with latent tabular diffusion models," in *Proceedings of the IEEE 40th International Conference on Data Engineering (ICDE)*, 2024, pp. 110–123.

[22] L. Chen, C. Wu, J. Li, S. Liu, G. Ye, and H. Chai, "TabLoft: Tabular data generation based on LLM with ordered features," in *Proceedings of the IEEE 42nd International Conference on Data Engineering (ICDE)*, 2026, pp. 1044–1056.

[23] J. Hyeong, J. Kim, N. Park, and S. Jajodia, "An empirical study on the membership inference attack against tabular data synthesis models," *arXiv preprint arXiv:2208.08114*, 2022.

[24] C. Ge, S. Mohapatra, X. He, and I. F. Ilyas, "Kamino: Constraint-aware differentially private data synthesis," *Proceedings of the VLDB Endowment*, vol. 14, no. 10, pp. 1886–1899, 2021.

[25] X. Chu, I. F. Ilyas, and P. Papotti, "Holistic data cleaning: Putting violations into context," in *Proceedings of the IEEE 29th International Conference on Data Engineering (ICDE)*, 2013, pp. 458–469.

[26] J. Qin, S. Huang, Y. Wang, J. Zhu, Y. Zhang, Y. Miao, R. Mao, M. Onizuka, and C. Xiao, "BClean: A Bayesian data cleaning system," in *Proceedings of the IEEE 40th International Conference on Data Engineering (ICDE)*, 2024, pp. 3407–3420.

[27] J. Rorseth, P. Godfrey, L. Golab, D. Srivastava, and J. Szlichta, "Recovering structure in unstructured LLM outputs," in *Proceedings of the IEEE 42nd International Conference on Data Engineering (ICDE)*, 2026, pp. 4329–4330.

[28] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *International Conference on Learning Representations*, 2023. [Online]. Available: https://openreview.net/forum?id=WE_vluYUL-X

[29] L. Wang, C. Ma, X. Feng, Z. Zhang, H. Yang, J. Zhang, Z. Chen, J. Tang, X. Chen, Y. Lin, W. X. Zhao, Z. Wei, and J.-R. Wen, "A survey on large language model based autonomous agents," *Frontiers of Computer Science*, vol. 18, no. 6, p. 186345, 2024.

[30] P. Li, X. Rao, J. Blase, Y. Zhang, X. Chu, and C. Zhang, "CleanML: A study for evaluating the impact of data cleaning on ML classification tasks," in *Proceedings of the IEEE 37th International Conference on Data Engineering (ICDE)*, 2021, pp. 13–24.

[31] S. Guha, F. A. Khan, J. Stoyanovich, and S. Schelter, "Automated data cleaning can hurt fairness in machine learning-based decision making," in *Proceedings of the IEEE 39th International Conference on Data Engineering (ICDE)*, 2023, pp. 3747–3754.

[32] D. Lopez-Paz and M. Oquab, "Revisiting classifier two-sample tests," in *International Conference on Learning Representations*, 2017. [Online]. Available: https://openreview.net/forum?id=SJkXfE5xx

[33] R. Shokri, M. Stronati, C. Song, and V. Shmatikov, "Membership inference attacks against machine learning models," in *2017 IEEE Symposium on Security and Privacy*, 2017, pp. 3–18.

[34] A. Machanavajjhala, J. Gehrke, D. Kifer, and M. Venkitasubramaniam, "ℓ-diversity: Privacy beyond k-anonymity," in *Proceedings of the 22nd International Conference on Data Engineering (ICDE)*, 2006, p. 24.

[35] N. Li, T. Li, and S. Venkatasubramanian, "t-closeness: Privacy beyond k-anonymity and ℓ-diversity," in *Proceedings of the IEEE 23rd International Conference on Data Engineering (ICDE)*, 2007, pp. 106–115.

[36] C. Dwork and A. Roth, *The Algorithmic Foundations of Differential Privacy*, ser. Foundations and Trends in Theoretical Computer Science. Now Publishers, 2014, vol. 9, no. 3–4.

[37] M. J. Sheller, B. Edwards, G. A. Reina, J. Martin, S. Pati, A. Kotrotsou, M. Milchenko, W. Xu, D. Marcus, R. R. Colen, and S. Bakas, "Federated learning in medicine: Facilitating multi-institutional collaborations without sharing patient data," *Scientific Reports*, vol. 10, p. 12598, 2020.

[38] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. Ag"uera y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics*, ser. Proceedings of Machine Learning Research, vol. 54. PMLR, 2017, pp. 1273–1282. [Online]. Available: https://proceedings.mlr.press/v54/mcmahan17a.html

[39] P. Kairouz, H. B. McMahan, B. Avent *et al.*, "Advances and open problems in federated learning," *Foundations and Trends in Machine Learning*, vol. 14, no. 1–2, pp. 1–210, 2021.

[40] K. Bonawitz, V. Ivanov, B. Kreuter, A. Marcedone, H. B. McMahan, S. Patel, D. Ramage, A. Segal, and K. Seth, "Practical secure aggregation for privacy-preserving machine learning," in *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, 2017, pp. 1175–1191.

[41] Q. Chen, Y. Chen, B. Yan, X. Jiang, X. Zhang, Y. Kang, T. Zhang, W. Huang, C. Gao, L. Fan, and Q. Yang, "Model trip: Enhancing privacy and fairness in model fusion across multi-federations for trustworthy global healthcare," in *Proceedings of the IEEE 40th International Conference on Data Engineering (ICDE)*, 2024.

[42] A. Janosi, W. Steinbrunn, M. Pfisterer, and R. Detrano, "Heart disease," UCI Machine Learning Repository, 1988.

[43] W. Wolberg, O. Mangasarian, N. Street, and W. Street, "Breast cancer wisconsin (diagnostic)," UCI Machine Learning Repository, 1993.

[44] "Heart failure clinical records," UCI Machine Learning Repository, 2020.

[45] B. V. Ramana, M. S. P. Babu, and N. B. Venkateswarlu, "ILPD (indian liver patient dataset)," UCI Machine Learning Repository, 2012.

[46] M. Little, "Parkinsons," UCI Machine Learning Repository, 2007.

[47] J. W. Smith, J. E. Everhart, W. C. Dickson, W. C. Knowler, and R. S. Johannes, "Using the ADAP learning algorithm to forecast the onset of diabetes mellitus," in *Proceedings of the Annual Symposium on Computer Application in Medical Care*, 1988, pp. 261–265.

[48] J. Demšar, "Statistical comparisons of classifiers over multiple data sets," *Journal of Machine Learning Research*, vol. 7, pp. 1–30, 2006.

[49] S. Holm, "A simple sequentially rejective multiple test procedure," *Scandinavian Journal of Statistics*, vol. 6, no. 2, pp. 65–70, 1979.